[Insertar logo institucional]

Servicio Nacional de Aprendizaje — SENA
Centro de Electricidad, Electrónica y Telecomunicaciones — CEET
Tecnólogo en Análisis y Desarrollo de Software (ADSO) · Trimestre 6

CAPÍTULO IV · DS-03

Especificación y Evidencia del Pipeline CI/CD

Integración, entrega y despliegue continuos

FASE: PLANEACIÓN / EJECUCIÓN

Proyecto formativo	EyeSchool — Sistema de control de asistencia escolar por QR y seguimiento académico
Ficha	3278641
Equipo / Autores	Andres Felipe Acero Lopez, Mariana Valentina Cortes Castillo, Samuel Stiven Sierra Rodriguez y Sergio Duvan Gomez Rodriguez
Instructor(es)	[Nombre del instructor técnico]
Criterio de evaluación	Trimestre 6 — En el proyecto se evidencia el despliegue del aplicativo según el diseño de la arquitectura en UML (diagrama de despliegue o diagrama de arquitectura de nube).
Versión del documento	0.2
Fecha	10/11/2026
Ciudad	Bogotá D.C.

Control del documento

Historial de versiones

Versión	Fecha	Autor(es)	Descripción del cambio
0.1	10/11/2026	Acero, Cortes, Sierra y Gomez	Creación del documento a partir de la plantilla
0.2	10/11/2026	Acero, Cortes, Sierra y Gomez	Pipelines reales de la API, la web y la app; evidencias y métricas DORA al 10/11/2026.
[1.0]			Versión aprobada

Revisión y aprobación

Rol	Nombre	Firma	Fecha
Elaboró (aprendiz / equipo)	Andres Felipe Acero Lopez, Mariana Valentina Cortes Castillo, Samuel Stiven Sierra Rodriguez y Sergio Duvan Gomez Rodriguez		
Revisó (líder técnico / Scrum Master)			
Aprobó (instructor / Product Owner)			

Convenciones de esta plantilla

[Texto resaltado]	Campo que el equipo debe diligenciar. Reemplace el texto y elimine el resaltado.
Instrucción	Orientación para diligenciar la sección. Elimine estos recuadros en la versión final.
DATOS DE EJEMPLO	Gráficas ilustrativas. Reemplácelas por las generadas con los datos reales del proyecto.

Ubicación en el paquete documental

Documento que lo precede	DS-02 Plan de instalación
Documento que lo sucede	DS-04 Manual de instalación
Norma / referente principal	Prácticas DevOps · ISO/IEC/IEEE 12207 · DORA metrics

Contenido

1. Propósito 4
2. Herramientas 4
3. Diseño del pipeline 4
4. Definición del pipeline (ejemplo) 5
5. Gestión de secretos 6
6. Evidencias de ejecución 6
7. Métricas DORA (opcional) 8

Si la tabla de contenido aparece vacía: clic derecho sobre ella → Actualizar campo → Actualizar toda la tabla.

1. Propósito

Documentar el pipeline que automatiza la construcción, verificación y despliegue del sistema, materializando en el entorno real lo diseñado en DS-01 y aplicando los quality gates de MC-01.

2. Herramientas

Plataforma CI/CD	GitHub Actions (CI de los tres repositorios, despliegue de la API y release de la app); Netlify y Vercel compilan y publican la web
Registro de artefactos	Artefactos de GitHub Actions (cobertura y JUnit) y GitHub Releases (APK). No se usan imágenes Docker: Azure App Service compila la API con Oryx
Análisis de calidad	SonarQube Cloud (API y web); flutter analyze (app)
Destino del despliegue	API: Azure App Service, QA apieyeschool-q (chilecentral) y producción api-eyesschool (westus3) · Web: Netlify (QA) y Vercel (producción) · App: APK en GitHub Releases con actualización OTA

3. Diseño del pipeline

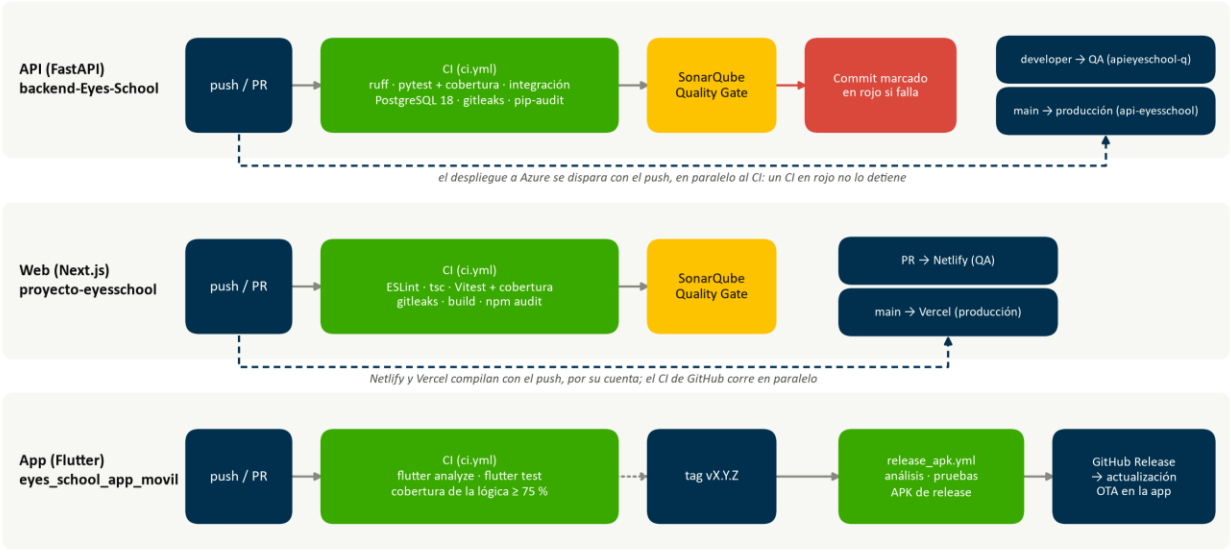


Figura 1. *Etapas reales del pipeline CI/CD de cada componente*

Tabla 1. *Especificación de etapas*

Etapa	Disparador	Acción	Criterio de éxito	Artefacto / salida
Lint y tipos	push / PR (los tres)	ruff (API) · ESLint y tsc (web) · flutter analyze (app)	0 hallazgos	Log del job
Pruebas unitarias	push / PR (los tres)	pytest · Vitest · flutter test, con cobertura	100 % en verde y cobertura ≥ umbral	coverage.xml, junit, lcov

Etapa	Disparador	Acción	Criterio de éxito	Artefacto / salida
Integración	push / PR (API)	pytest -m integration contra un servicio postgres:18 desechable	100 % en verde	junit-integracion.xml
Secretos y dependencias	push / PR (API y web)	gitleaks · pip-audit · npm audit	0 secretos · 0 vulnerabilidades altas	Log del job
Análisis estático	push / PR (API y web)	SonarQube Cloud	Quality Gate aprobado	Panel de SonarQube
Build	push / PR (web) · tag (app)	next build · flutter build apk	Compila sin errores	.next · app-release.apk
Despliegue QA	push a developer (API) · PR (web)	Azure App Service apieyeschool-q · Netlify deploy preview	Despliegue exitoso; humo manual GET /cursos 200 (no hay /health)	URL de QA
Despliegue producción	push a main (API y web) · tag vX.Y.Z (app)	Azure api-eyesschool · Vercel · GitHub Release	Despliegue exitoso; sin aprobación manual ni humo automático	Release / URL de producción

Diferencia con la plantilla: en la API y la web el despliegue no espera al CI. El workflow de Azure, Netlify y Vercel se disparan con el mismo push, así que un Quality Gate en rojo marca el commit pero no lo detiene. El 10/11/2026 las primeras 17 corridas del CI de la API fallaron y las 31 publicaciones en QA se hicieron igual.

4. Definición del pipeline (ejemplo)

```yaml
.github/workflows/ci.yml — API, rama developer (extracto)
name: CI
on:
  pull_request:
  push:
    branches: [develop, developer, main]
jobs:
  lint_test_audit:
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: actions/setup-python@v5
        with: { python-version: '3.11', cache: pip }
      - run: ruff check app main.py tests scripts
      - run: pytest -q -m "not integration" --cov=app --cov-branch --cov-report=xml
      - run: pip-audit -r requirements.txt
  integracion:
    services:
      postgres: { image: postgres:18 }
    steps:
      - run: pytest -m integration --cov=app --cov-report=xml:coverage-integracion.xml
  secretos:
    steps:
      - uses: gitleaks/gitleaks-action@v2
  sonarqube:
    needs: [lint_test_audit, integracion]
    steps:
      - uses: SonarSource/sonarqube-scan-action@v5
```

```
        env: { SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }} }
```

Los demás pipelines: el ci.yml de la web (PR #34) corre npm ci, eslint --max-warnings=0, tsc, Vitest con cobertura, gitleaks, SonarQube, next build y npm audit. El ci.yml de la app (PR #15) corre flutter analyze, flutter test --coverage y la cobertura por capa; release_apk.yml compila el APK al crear un tag. Los despliegues de la API (developer_apieyeschool-q.yml y main_api-eyesschool.yml) los generó Azure: instalan requirements.txt, suben el código y lo publican con azure/webapps-deploy.

5. Gestión de secretos

Secreto	Almacenado en	Usado por
SONAR_TOKEN	GitHub Secrets (API, web y app)	Job de SonarQube Cloud
Cadena de conexión BD (DATABASE_URL)	Configuración de cada Web App en Azure. En GitHub existe además como variable del repositorio de la API, no como secreto: moverla a Secrets	La API en ejecución. El CI usa una base desechable postgres:18
Keystore Android	No existe: el APK de release se firma con la llave de depuración (build.gradle.kts)	release_apk.yml (pendiente: keystore propio en Secrets)
Credenciales de Azure (client, tenant y subscription ID, con OIDC)	GitHub Secrets (API y web)	azure/login en los despliegues. La API conserva 6 juegos de workflows retirados: limpiar los que sobran
QA_PASSWORD	GitHub Secrets (los tres) y canal privado del equipo	Pruebas contra QA: Newman, k6 y Playwright
NEXT_PUBLIC_API_URL	GitHub Secrets y variables de Netlify y Vercel	Build de la web (es pública por diseño)
GITHUB_TOKEN	Automático de GitHub Actions	gitleaks y creación de la release del APK

6. Evidencias de ejecución

Capturas generadas con GitHub CLI (gh) sobre las corridas reales del 10/11/2026 y la release de la app.

```
$ gh run view 42899050528 -R AndresFelipeAceroLopez/backend-Eyes-School
✓ developer CI AndresFelipeAceroLopez/backend-Eyes-School#16 · 42899050528
Triggered via push · 2026-11-10 20:49 UTC
JOBS
✓ Lint, pruebas y auditoria in 59s
✓ Deteccion de secretos (gitleaks) in 6s
✓ Pruebas de integracion (PostgreSQL 18) in 57s
✓ SonarQube Cloud (Quality Gate) in 45s
```

Figura 2. *Ejecución exitosa completa del CI de la API (cuatro jobs en verde)*

```
$ gh run view 42884643243 -R AndresFelipeAceroLopez/backend-Eyes-School
X developer CI AndresFelipeAceroLopez/backend-Eyes-School#16 · 42884643243
Triggered via push · 2026-11-10 19:22 UTC
JOBS
✓ Lint, pruebas y auditoria in 1m7s
✓ Pruebas de integracion (PostgreSQL 18) in 2m7s
✓ Deteccion de secretos (gitleaks) in 5s
X SonarQube Cloud (Quality Gate) in 18s
  ✓ Set up job
  ✓ Run actions/checkout@v4
  ✓ Descargar cobertura de las unitarias
  ✓ Descargar cobertura de las de integracion
  X SonarQube Cloud
  ✓ Post SonarQube Cloud
  ✓ Post Run actions/checkout@v4
$ gh run view 42884643243 --log-failed | grep ERROR
ERROR You are running CI analysis while Automatic Analysis is enabled.
INFO  EXECUTION FAILURE
##[error]Process completed with exit code 3.
```

Figura 3. *Ejecución detenida por el Quality Gate: SonarQube falla porque el análisis automático estaba activado*

```
$ gh run list -R AndresFelipeAceroLopez/backend-Eyes-School --limit 12
✓  11-10 20:49  Deploy apieyeschool-q   developer                test(carga): S2 tras optimizar el pool: la mitad
✓  11-10 20:49  CI                      developer                test(carga): S2 tras optimizar el pool: la mitad
✓  11-10 20:39  Deploy apieyeschool-q   developer                docs(qa): los 23 defectos creados como issues de
✓  11-10 20:39  CI                      developer                docs(qa): los 23 defectos creados como issues de
✓  11-10 20:34  CI                      developer                fix(seguridad): cabeceras que marco OWASP ZAP (C
✓  11-10 20:34  Deploy apieyeschool-q   developer                fix(seguridad): cabeceras que marco OWASP ZAP (C
✓  11-10 20:30  CI                      developer                docs(qa): DEF-022 medido y mejorado un 65 % (866
✓  11-10 20:30  Deploy apieyeschool-q   developer                docs(qa): DEF-022 medido y mejorado un 65 % (866
✓  11-10 20:25  Deploy apieyeschool-q   developer                perf(db): dimensionar el pool con los tiempos re
✓  11-10 20:25  CI                      developer                perf(db): dimensionar el pool con los tiempos re
✓  11-10 20:15  Deploy apieyeschool-q   developer                docs(qa): la gestion de defectos pasa a GitHub I
✓  11-10 20:15  CI                      developer                docs(qa): la gestion de defectos pasa a GitHub I

$ gh run list -R marianacortes0/proyecto-eyesschool --workflow CI --limit 3
✓  11-05 21:02  CI                      feat/web-calidad-pruebas  Feat/web calidad pruebas
✓  11-05 21:01  CI                      feat/web-calidad-pruebas  Feat/web calidad pruebas
✓  11-03 15:53  CI                      feat/web-calidad-pruebas  Feat/web calidad pruebas

$ gh run list -R AndresFelipeAceroLopez/eyes_school_app_movil --limit 5
✓  10-25 23:55  CI                      chore/modelo-calidad      Modelo de calidad: CI, pruebas de repositorios,
✓  10-25 22:08  CI                      chore/modelo-calidad      Modelo de calidad: CI, pruebas de repositorios,
✓  10-25 22:05  CI                      chore/modelo-calidad      Modelo de calidad: CI, pruebas de repositorios,
✓  09-17 20:59  Release APK             v1.0.1                    fix(ci): add write permissions for github token
X  09-17 20:51  Release APK             v1.0.1                    feat(ota): add github actions and auto-update me
```

Figura 4. *Historial de ejecuciones de los tres repositorios*

```
$ gh release view v1.0.1 -R AndresFelipeAceroLopez/eyes_school_app_movil
title:  Release v1.0.1
tag:  v1.0.1
author:  github-actions[bot]
created:  2026-09-17T20:59:04Z
published:  2026-09-17T21:14:07Z
url:  https://github.com/AndresFelipeAceroLopez/eyes_school_app_movil/releases/tag/v1.0.1
asset:  app-release.apk
--

$ gh run list -R AndresFelipeAceroLopez/eyes_school_app_movil --workflow "Release APK"
✓  09-17 20:59  Release APK             v1.0.1                    fix(ci): add write permissions for github token
X  09-17 20:51  Release APK             v1.0.1                    feat(ota): add github actions and auto-update me
```

Figura 5. *Release v1.0.1 de la app publicada por release_apk.yml a partir del tag*

Lectura de las evidencias: las 17 primeras corridas del CI de la API en developer fallaron todas por la misma configuración de SonarQube (el análisis automático estaba activado a la vez que el del CI), no por el código;

desde la corrida 18 el CI queda en verde. En la web, las 8 corridas del CI pasaron y el Quality Gate de SonarQube está aprobado. En la app, las 3 corridas del CI pasaron y la primera ejecución de release_apk.yml falló por permisos del token, corregidos en la segunda.

7. Métricas DORA (opcional)

Métrica	Definición	Valor
Frecuencia de despliegue	Despliegues a producción por semana	API: 0,7 por semana (14 versiones distintas entre el 17/06 y el 10/11/2026). Web: 3 en junio (Vercel). App: 1 release (v1.0.1). En QA, la API se desplegó 31 veces entre el 02/11 y el 10/11
Tiempo de entrega de cambios	Del commit a producción	API: mediana de unos 5 días (123 h) entre el commit y su despliegue en producción. Es aproximado: el historial se reescribió el 09/11/2026
Tasa de fallo de cambios	Despliegues que causan incidente ÷ total	Sin registro de incidentes para calcularla. Despliegues de producción fallidos: 3 de 19 (16 %). Incidente conocido: el inicio del estudiante respondía 500 en producción (DEF-002), corregido con el hotfix del PR #20
Tiempo de recuperación	Tiempo medio para restaurar el servicio	No medible: no hay monitoreo ni registro de la hora de detección. El hotfix de DEF-002 llegó a producción el 10/11/2026

Acciones de mejora del pipeline: (1) que el despliegue de la API dependa del CI (workflow_run o un job con needs) y activar las reglas de protección de developer, develop y main, hoy desactivadas en la API y la web; (2) firmar el APK con un keystore propio guardado en Secrets; (3) agregar un endpoint /health y una prueba de humo tras cada despliegue; (4) mover DATABASE_URL de variable a secreto en GitHub; (5) actualizar sonarqube-scan-action a v6, porque GitHub avisa que la v5 tiene una vulnerabilidad; (6) fusionar los PR #15 y #34 para que el CI de la app y de la web corra también en sus ramas de integración.

[Nombre completo]	[Nombre completo]
Elaboró	**Revisó**
Fecha: [dd/mm/aaaa]	Fecha: [dd/mm/aaaa]